Chapter 20

DBMS

Transactions

- A transaction in DBMS is a set of one or more operations executed as a single unit of work, which must be completed entirely or not executed at all.

A transaction is a complete task that must either:

- Fully succeed, or
- Fully fail (rollback)

No partial execution is allowed.

- **Commit**
- **Rollback**

Properties of Transaction (ACID)

- **Atomicity** (All or Nothing)
Either the whole transaction happens or none.

- **Consistency**
Database remains valid before and after transaction.

- **Isolation**
Transactions do not interfere with each other.

- **Durability**
Once committed, changes are permanent even if system crashes.

Schedule

- A schedule is a sequence of read, write, commit, and rollback operations from one or more transactions.

Concurrency

- Concurrency is the simultaneous execution of multiple transactions to improve system performance and resource usage.

Example:

Step	T1	T2
1	Read(A)	
2	Read(B)	
3		Read(A)
4		A = A − 10
5	Write(C)	
6		Write(A)
7	A = A + 10	
8	Write(A)	

Why Concurrency?

- Better performance (throughput).
- Faster response time.
- Efficient resource utilization.

Concurrency Problems

- Recoverability problem.
- Deadlock.
- Serializability issues.

Dirty Read Problem or temporary update

Step	T1 (Transaction 1)	T2 (Transaction 2)
1	Read(A = 100)	
2	A = A + 50 → 150	
3	Write(A = 150)	
4		Read(A = 150)
5	**failed/Rollback (A → 100)**	
6		Uses A = 150 (WRONG)

Lost Update:

Step	T1 (Transaction 1)	T2 (Transaction 2)
1	R(X) = 5	
2	X = X + 2 → 7	
3	W(X) = 7	
4		W(X) = 10
5		Commit (success)
6	Commit (fails)	

Good Schedule:

- A good schedule is one that gives the same result as some serial execution (i.e., transactions run one after another).

Incorrect Summary Problem Table

Step	T1 (Transaction 1)	T2 (Transaction 2)
1	R(X) = 5	
2	X = X + 10 → 15	
3	W(X)	
4		R(X) = 15
5		R(Y) = 10
6		Sum = X + Y = 25
7		W(Sum)
8	R(Y) = 10	
9	Y = Y + 10 → 20	
10	W(Y)	

Serial vs Non-Serial Schedule.

Step	T1 (Transaction 1)	T2 (Transaction 2)
1	R(X)=100	
2	X = X + 10 → 110	
3	W(X)=110	
4		R(X)=110
5		X = X − 20 → 90
6		W(X)=90

Serializable schedule.

- In database concurrency control, a schedule is called serializable if the result of executing multiple transactions concurrently is equivalent to some serial (one-by-one) execution of those same transactions.

T1	**T2**
R(x)	
x = x + 5	
W(x)	
	R(x)
	x = x + 10
	W(x)
R(y)	
y = y – 2	
W(y)	
	R(y)
	y = y – 3
	W(y)

Calculating Total Serial Schedules.

- If Schedule S has three transactions t1, t2, t3

Total possible serial schedules 3! = 6.

Conflict.

Two operations are in conflict if:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

Conflict Serializability.

- Conflict Serializability.
- View Serializability.

Conflict Equivalent

T1	T2
R(X)	
	W(X)
	R(Y)
R(Z)	
	W(Z)
R(Y)	

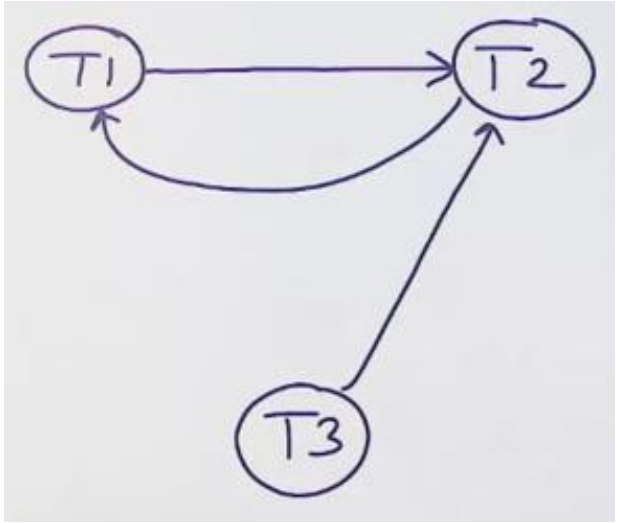
T1	T2
R(X)	
	W(X)
R(Z)	
	R(Y)
R(Y)	
	W(Z)

Conflict Serializability.

- To prove a given schedule conflict Serializability.
- We will use graphs.

Conflict Serializability.

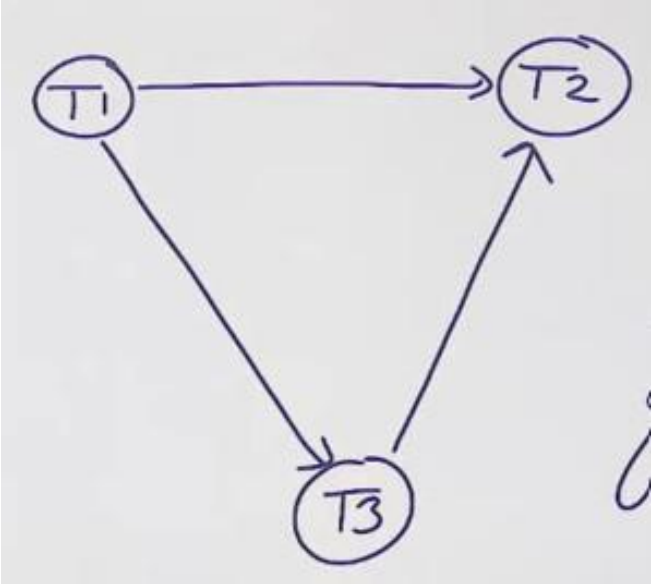
T1	T2	T3
R(X)		
	W(X)	
		R(Y)
	W(Y)	
R(Y)		



NOT CONFLICT SERIALIZABILITY

Conflict Serializability.

T1	T2	T3
READ(X)		
	READ(Y)	
		READ(Y)
	WRITE(Y)	
WRITE(X)		
		WRITE(X)
	READ(X)	
	READ(X)	



Serial Schedule = T1 -> T3 -> T2

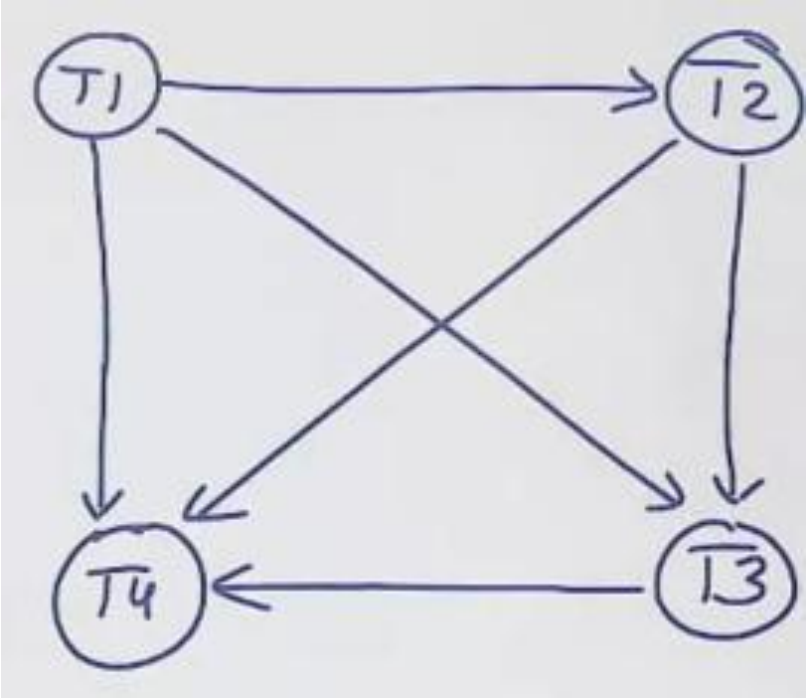
Conflict Serializability.

Consider the following schedules for transactions T1, T2, T3 AND T4. R1A, W1B, R2B, W1A, R4A, R2C, W4A, W3B, R4B, W4C

The given schedule is serializable or not?

Conflict Serializability.

T1	T2	T3	T4
R(A)			
W(B)			
	R(B)		
		R(C)	
W(A)			
			R(A)
	R(C)		
			W(A)
		W(B)	
			R(B)
			W(C)



T1->T2->T3->T4

View Serializability.

- A schedule is called view serializable if it is view-equivalent to some serial schedule.

Three Points:

- Who reads from database
- Who reads from others written value
- Who writes last.

View Serializability.

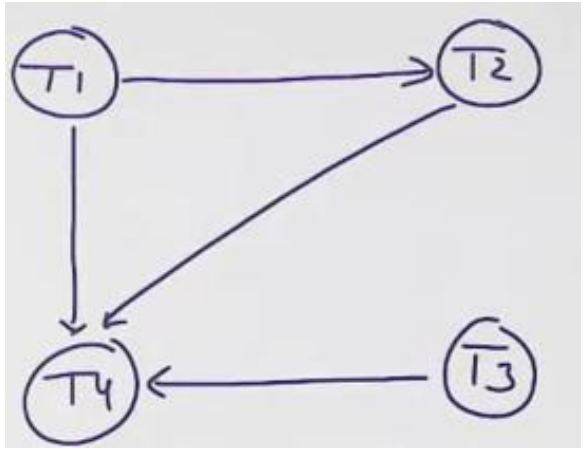
T1	T2	T3
R(X)		
W(X)		
	R(X)	
		W(X)
	W(X)	

Is not Equal

T1	T2	T3
R(X)		
	R(X)	
W(X)		
	W(X)	
		W(X)

View Serializability.

T1	T2	T3	T4
W(X)			
	R(X)		
		W(X)	
	W(X)		
			W(X)



T1 , T2 , T3 , T4

T1 , T3 , T2 , T4

T3 , T1 , T2 , T4

View Serializability.

T1	T2	T3	T4
W(X)			
	R(X)		
	W(X)		
		W(X)	
			W(X)

EQUALIZE

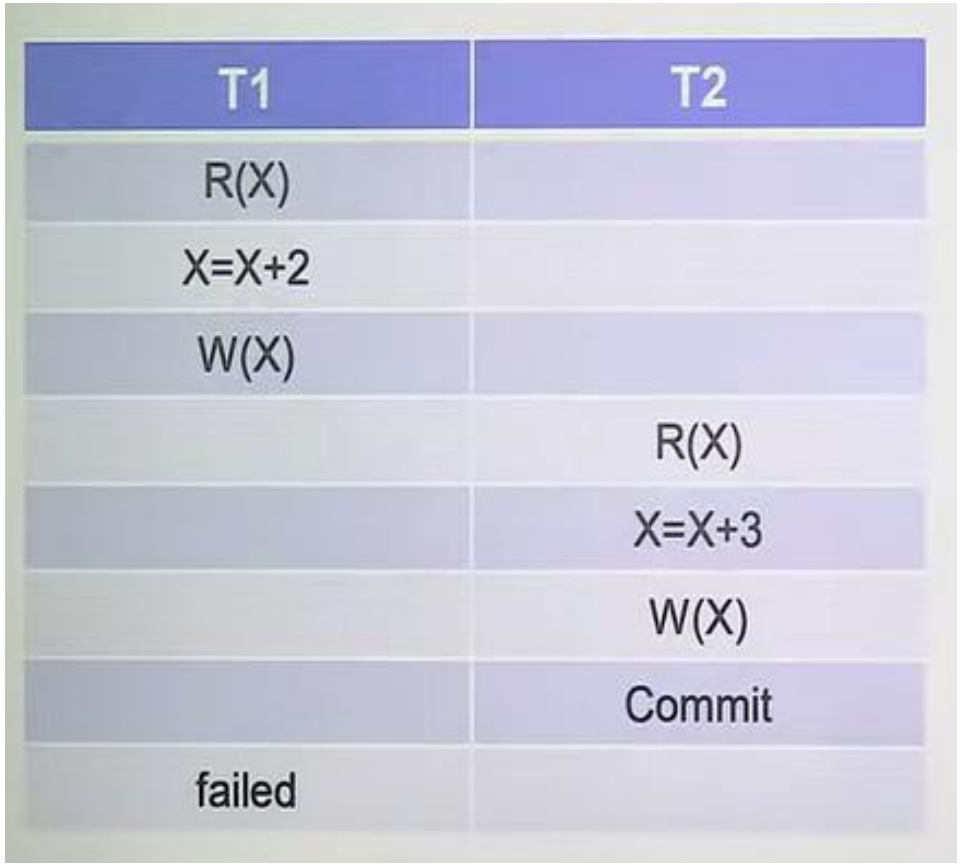
T1	T2	T3	T4
W(X)			
		W(X)	
	R(X)		
	W(X)		
			W(X)

NOT EQUALIZE

T1	T2	T3	T4
		W(X)	
W(X)			
	R(X)		
	W(X)		
			W(X)

EQUALIZE

Recoverability.



T1	T2
R(X)	
X=X+2	
W(X)	
	R(X)
	X=X+3
	W(X)
	Commit
failed	

Recoverable Schedule

T1	T2
R(X)	
X=X+2	
W(X)	
	R(X)
	X=X+3
	W(X)
Commit	
	Commit

Rollback types

- Cascading Rollback
- No Cascading Rollback

Cascading Recoverable Rollback

T1	T2
R(X)	
X=X+2	
W(X)	
	R(X)
	X=X+3
	W(X)
Commit	
	Commit

No Cascading Recoverable Rollback

T1	T2
R(X)	
X=X+2	
W(X)	
Commit	
	R(X)
	X=X+3
	W(X)
	Commit